```
library(DBI)

library(RSQLite)


# When running this line, a window will pop up.

# Select 'project1_raw_data.db' from wherever it is saved on your computer!

sales <- dbConnect(SQLite(), dbname = file.choose())


# List all tables to verify successful connection

dbListTables(sales)



##SECTION A##

#Step 1

SQL_train_head <- dbGetQuery(sales, "SELECT *

                     FROM train

                     LIMIT 5")

top_3_products <- dbGetQuery(sales, "SELECT item_nbr, SUM(units)

                    FROM train

                    GROUP BY item_nbr

                    ORDER BY SUM(units) DESC

                    LIMIT 3")

#step 2- I replaced 'ON' with 'USING' in the join as I kept getting duplicate columns

#The code here will be repeated in step 3 as I cannot reference 'joined tables' within my SQL statements
```

#To ensure that my main table for analysis is relevant to the business question I will filter the table to only show the values relevant to the top 3 products(5, 9 and 45)

```
joined_tables <- dbGetQuery(sales, "
                    SELECT *
                    FROM train
                    INNER JOIN key
                      USING (store_nbr)
                    INNER JOIN weather
                      USING (station_nbr, date)
                    WHERE item_nbr IN (45, 9 ,5)
                    ")



View(joined_tables)
```

#task 3- for this task I will reference a top product using its ID(5)

```
weather_head <- dbGetQuery(sales, "SELECT *
                     FROM weather
                     LIMIT 5")
```

#Select statement only includes columns relevant to the task

```
sales_temp_5 <- dbGetQuery(sales, "SELECT store_nbr, station_nbr, date, item_nbr, units, tavg
                    FROM train
                    INNER JOIN key
```

```
                    USING (store_nbr)
                  INNER JOIN weather
                    USING (station_nbr, date)
                WHERE item_nbr = 5
                ORDER BY units DESC
                ")
View(sales_temp_5)



##SECTION B##
joined_tables
library(readr)
library(dplyr)
library(tidyr)

write_csv(joined_tables, 'joined_tables.csv')

head(joined_tables)
summary(joined_tables)

#task 1- converting data types

joined_tables <- joined_tables %>%
  mutate(date = as.Date(date))
```

#The task only mentions changing the 'Date' column, but 'sunrise' and 'sunset' are in character when they should be in time.

joined_tables$sunrise <- as.POSIXct(joined_tables$sunrise, format = "%H:%M:%S")

joined_tables$sunset <- as.POSIXct(joined_tables$sunset, format = "%H:%M:%S")

#POSIXct is a date-time class used when you only have the time available in a column

str(joined_tables)

#task 2- cleaning data

summary(joined_tables)

#The summary stats here shows that the majority of columns contain NA values.

#I will replace the NA values with the median(instead of mean).

#This is because the median is not influenced by outliers(unlike the mean).

joined_tables <- joined_tables %>%

  mutate(across(c(tmax, tmin, tavg, depart, dewpoint, wetbulb, heat, cool, sunrise, sunset, snowfall, preciptotal, stnpressure, sealevel, resultspeed, resultdir, avgspeed), ~replace_na(., median(., na.rm=TRUE))))

summary(joined_tables)

#Now there are no NA values in any integer columns!

#removing outliers in units_sold

```
library(ggplot2)


#creating box plots to view outliers

units_bxplot <- joined_tables %>%

  ggplot(aes(units, units)) +

  geom_boxplot()
```

#The box plot shows that my data is quite skewed, I shall perform a log transformation on it and use that column in predictive modelling.

```
joined_tables$units_log <- log(joined_tables$units + 1)


units_log_bxplot <- joined_tables %>%

  ggplot(aes(units_log, units_log)) +

  geom_boxplot()
```

#Now that I have applied log transformation I can better see the box plot. In terms of analysis I will be using this column going forward.

#Log transformed regression works on percentages/multipliers.

#3- Feature Creation

#Label Encoding (for Binary Categories: Weekday or Weekend)

#creating is_weekend will take a few steps, I will have to start with extracting the day of the week from the date column, I will need the lubricate package to do this.

```
library(lubridate)


joined_tables <- joined_tables %>%
  mutate(weekday_number = wday(date, week_start = 1))


colnames(joined_tables)

# now there is a column containing the day of the week I can add label encoding


joined_tables$is_weekend <- ifelse(joined_tables$weekday_number == 6 | joined_tables$weekday_number == 7, 1, 0)

colnames(joined_tables)

#This shows that I have now made a column called is_weekend where the result is 1 when it is a weekend and 0 when it is not.

#This will be useful as the day of the week influences whether people buy certain products, some products may be bought more on the weekends as people are not in work so they have time to go to the shops, or because the item is utilized more on weekends(like game controllers as an example).


#Label Encoding (for Binary Categories: raining or not raining)

#Because codesum uses character strings, I am unable to use == like before, I have to use a new function str_detect here instead

joined_tables <- joined_tables %>%
  mutate(is_raining = ifelse(stringr::str_detect(codesum, "RA|DZ|TS"), 1, 0))

head(joined_tables$is_raining)

#The data shows that the majority of days with units_sold are not raining, this will be useful to plot on a graph to show whether rain impacts the sales of the top 3 products.
```

```
#To create a seasonality column I will need to make a column showing the month number for each date.

joined_tables <- joined_tables %>%

  mutate(month_number = month(date))

head(joined_tables$month_number)

#now I can separate the months into the 4 seasons.

#I originally indented to use binning here but I discovered that it would not work.

#The seasons do not follow a regular pattern(the last month is winter as well as the first 2 months).

#Because of this I decided to use case_when and make vectors inside the function that follow the seasonal patterns.

joined_tables <- joined_tables %>%

  mutate(season = case_when(

    month_number %in% c(12,1,2) ~ "Winter",

    month_number %in% c(3,4,5) ~ "Spring",

    month_number %in% c(6,7,8) ~ "Summer",

    month_number %in% c(9,10,11) ~ "Autumn"))

head(joined_tables$season)

#this will be useful when determining how seasons impact the units_sold, perhaps certain products would be bought more in the Winter as they are better suited to the winter(like a jacket).



#Creating Predictive models
```

```
# specifying 60/40 split

sample <- sample(c(TRUE, FALSE), nrow(joined_tables), replace = T, prob = c(0.6,0.4))

# subset data points into train and test sets

train <- joined_tables[sample, ]

test <- joined_tables[!sample, ]


#Fitting multiple Linear Regression Model


mlr_model <- lm(units_log ~ is_weekend + is_raining + month_number, data=train)

mlr_model


#Residual Standard Error(RSE)

sigma(mlr_model)

#RSE(log) = 1.92

#because I used the log column I will convert this number exponentially

exp(1.920862)

#RSE = 6.8

#The value 6.8 means that my model's predictions are (on average) within a factor of 6.8 of the actual sales.

#This is quite a wide prediction range. I will complete R2 and compare.



summary(mlr_model)$r.squared
```

#r.squared = 0.00198

#The value of 0.00198 means that my weekend and weather features account for 0.2% of the variance in units sold.

#This is an incredibly low value, hinting that variables like rain and weekends vs weekdays and month number have a minimal impact of the variance in sales.

#This heavily suggests that the top 3 items are more impacted by other unmeasured factors- such as temperature.

#interpreting key model outputs in linear regression model

summary(mlr_model)

#is_weekend(estimate)= 0.10

#When holding all variables constant, the weekends see a 10% increase in units sold compared to weekdays. This makes sense as people would have more free time on the weekends and therefore can purchase more items in the store.

#is_raining= -0.14(2DP)

#When holding all variables constant, rainy days result in a 14% decrease in units sold. This makes sense as less people would be inclined to go into our stores when the weather is bad

#month_number= -0.0036

#Progessing through each month in the years, the amount of units sold change by 0.36%. This perhaps shows that seasonality does not have a significant impact on consumers purchasing out items.

plot(mlr_model, which = 1)

#This plot predicts sales in three distinct categorical blocks(is_raining, is_weekend and month_number)

#The red line is flat and it is very close to 0, this indicates that the assumption of linearity holds true and there is not a hidden pattern that is non-linear(one of the requirements for linear regression is data being linear).

#The 3 vertical spreads of residuals highlights that the model has high levels of bias, the features I made do not have enough detail to explain the variations in units_sold

#It is evident here that since my features are categorical, the linear model only predicted the groups averages.

#Logistic Regression#

Creating a Binary Target Variable.1= High sales day, 0=low sales day.

```
benchmark <- median(train$units_log, na.rm=TRUE)

train$high_sales <- ifelse(train$units_log > benchmark, 1, 0)

test$high_sales <- ifelse(test$units_log > benchmark, 1, 0)
```

#Training the model- similar to linear regression in terms of syntax

```
logistic_model <- glm(high_sales ~ is_weekend + is_raining + month_number, data=train, family='binomial')
```

#checking the coefficients and p-values

```
summary(logistic_model)
```

#before I plot the ROC curve I need to import relevant libraries.

```
if(!require(pROC)) install.packages("pROC")

library(pROC)
```

```
table(test$high_sales)


#Making probabilites on the TEST data

test_prob <- predict(logistic_model, newdata = test, type='response')

ROC_score <- roc(test$high_sales, test_prob)

plot(ROC_score, main='ROC curve for predicting High Sales',col='#ADD8E6',
lwd=3, print.auc=TRUE)


#Despite this being a graph for a ROC curve my graph is straight, this now
confirms the analysis in my earlier linear regression model.

#Unfortunately an AUC of 0.51 means that my model performs no better than
random guessing.

#If I combine this with my low R2 value, this proves that external
environmental factors(weather) and time indicators(weekends,months) do not
significantly drive the sales variance for the top products.

#This now suggests that future modelling should pivot to more statistical
business metrics like price fluctations



#To bridge the gap between data engineering and business intelligence my
finalized dataset will be exported from R as a CSV and imported into Power BI.

#This will allow me to build an interactive retail dashboard.

#I will attach a page filter using the item_num column so stakeholders can
toggle between the top 3 products.
```

```
#exports
write.csv(joined_tables, "cleaned_retail_data.csv", row.names = FALSE)
getwd()

#Adding this so I dont get a dbDisconnect error
dbDisconnect(sales)
```